Instituto Superior de Formación Técnica Nº 151
Carrera: Analista en Sistemas
3 Año. Algoritmos y Estructuras de Datos III.
Alumna: LUZURIAGA, Silvana

RECURSION.

Marco Teórico:

Responder el siguiente cuestionario en función de la bibliografía Obligatoria.

1. Que entiende por Recursión
Es una técnica de programación donde un **método o función se llama a si mismo** para resolver un problema. La gran idea detrás de esto es la estrategia de **"Dividir y Conquistar".** Tomamos un problema complejo y lo dividimos en subproblemas mas chicos y de la misma naturaleza hasta que llegamos a un punto lo suficientemente simple como para resolverlo directamente.

2. Que es una función Recursiva y que entiende por Recurrencia , dar ejemplos.
Para que un algoritmo recursivo sea correcto y no rompa la memoria del sistema, necesita si o si dos partes fundamentales:
* **Caso Base** (o Condición de Parada): es el escenario básico que se puede resolver directamente sin necesidad de volver a hacer otra llamada Recursiva. Es lo que frena el bucle de llamadas. Si no existiera o estuviera mal programado, el programa se ejecutaría infinitamente.
* **Caso Recursivo**: es la parte donde la función se llama a si misma, pero siempre modificando el parámetro de entrada para acercarse un paso mas hacia el caso base.

3. Explicar le uso del Stack del sistema operativo en Funciones Recursivas, dar un ejemplo.
A nivel de hardware, cada vez que una función Recursiva se llama a si misma, el sistema operativo no destruye la función anterior; en su lugar, guarda su estado actual en una estructura de datos en memoria llamada **Pila de Ejecución (Call Stack).**
Cada llamada nueva se "apila" arriba de la anterior. Cuando por fin se alcanza el caso base, la pila empieza a "desapilarse" resolviendo las operaciones de arriba hacia abajo hasta llegar a la llamada inicial. Por esta razón, la Recursión suele consumir mas complejidad espacial (memoria) que un bucle iterativo común.

4. Explicar las distintas variantes de Recursión.
Dependiendo de cómo y dónde se realice la llamada dentro del código, la recursion se clasifica en diferentes variantes:
***Recursión Lineal (o Simple)**: Es la mas común. Ocurre cuando la función se llama a si misma una sola vez dentro del caso recursivo (por ejemplo, el calculo del Factorial, donde solo llamamos a **factorial (n-1)**).
***Recursion Múltiple:** se da cuando el caso recursivo contiene dos o mas llamadas a la propia función de manera interna. El ejemplo tipico es **Fibonacci**, donde para calcular un numero necesitas invocar en la misma linea a **fibonacci(n-1) + fibonacci(n-2).**

Recursion Mutua(o Cruzada): Se presenta cuando tenes dos funciones (por ejemplo, **Función A y Función B**) donde la **Funcion A** en su lógica llama a la **Función B, y la Función B** a su vez llama a la **Función A**. Se va turnando entre si hasta alcanzar un caso base comun.

5. Describir las diferencias entre Iteración y Recursión.
Aunque ambas técnicas sirven para repetir un bloque de instrucciones hasta que se cumpla una condición, lo hacen de formas muy distintas a nivel de logica y de hardware.

Caracteristica	**Iteracion(Bucles for/while)**	**Recursion (llamadas a si misma)**
Mecanismo	Utilliza estructuras de control repetitivas expresas.	Utiliza llamadas repetidas a funciones sobre si mismas.
Condición de fin	Termina cuando la condición del bucle se vuelve falsa.	Termina cuando se alcanza explicitamente el **Caso Base**.
Riesgo de ciclo infinito	Si la condición nunca cambia, el bucle se ejecuta infinitamente consumiendo CPU, pero no rompe la memoria.	Si falta el caso base, ocurre un **Stack Overflow** (se llena la pila de ejecución y el programa crashea).
Legibilidad	El código puede volverse mas largo y complejo de leer en problemas lógicos complejos.	El código suele ser mucho mas corto, elegante y fiel al pensamiento matemático (“Dividir y Conquistar”)

Marco Practico:

1. Para el Ejercicio de la Unidad anterior: “*Un Negocio necesita gestionar la atención a sus cliente, el mismo recibe los clientes en un Box de Atencia que registra los clientes y los carga para su atención según su llegada. Nos piden desarrollar un sistema de “Turnos” que se muestren en una pantalla.*
La aplicación deberá registrar los clientes, mostrarlos en orden de llegada, llamarlos según ese orden por pantalla con una opción “próximo cliente – Box nro”
Nos Solicitan imprimir el tiempo de “Espera” de cada persona en la Lista, deberá Implementar esto de Forma recursiva.

   El problema nos pide gestionar clientes en orden de llegada (lo que estructuralmente es una **Cola - FIFO: First In, First Out)**. Sin embargo, la consigna especifica requiere implementar de **forma recursiva** la función **para imprimir el tiempo de espera de cada persona en la lista.**
   Para calcular el tiempo de espera acumulado de cada cliente de manera recursiva, pensamos bajo la estrategia de “Dividir y Conquistar”.
   ***Caso Base**: si llegamos al principio de la cola(primer cliente), su tiempo de espera acumulado es simplemente su propio tiempo de atencion.

***Caso Recursivo**: El tiempo de espera de un cliente i es igual a su propio tiempo de atención mas el tiempo de espera acumulado de todas las personas que tiene adelante en la fila.

2. Escribir dos funciones mutuamente recursivas par(n) e impar(n) que determinen la paridad del numero natural dado, conociendo solo que:

    a. 1 es impar.

    b. Si un número es impar, su antecesor es par; y viceversa.

Este ejercicio es un claro ejemplo de la Recursion Mutua o Cruzada(donde una función invoca a la otra y viceversa de manera alternada).
Basándonos estrictamente en las reglas:
*** Caso Base**: sabemos que **1** es impar de forma directa. Por ende, si evaluamos la paridad en **1** dentro de **impar(1) es verdadero**, y en **par (1) es falso.** Para complementar el ciclo matemático de números naturales, si llegamos a **0**, sabemos que **0** es par.
***Caso Recursivo**: para saber si un número **n** es par, le preguntamos a la función impar si su antecesor **(n - 1) es impar.** Para saber si es impar, le preguntamos a la función par si su antecesor **(n - 1) es par.**

COMPARTO EL LINK DEL REPOSITORIO CON LOS EJERCICIOS RESUELTOS:
https://github.com/Sylvana74/Algoritmos_Estructuras_De_Datos_3_2026/tree/main

Lic. Oemig José Luis.